STAYEASY
HÉBERGEMENT & COLOCATION

esprit
Se former autrement

StayEasy

Réserver un séjour.
Trouver une colocation.

RÉSERVER

Chambres et séjours

PARTAGER

Annonces de colocation

NOTRE ÉQUIPE

Isra Khlifi

Molka Kammoun

Lina Kazdaghli

Mohamed Khouni

Samar Masmoudi

Nermine Karoui

LE FIL DE LA PRÉSENTATION

SOMMAIRE

01 Présentation du projet et problématique

02 Acteurs et fonctionnalités principales

03 Architecture globale et microservices

04 Technologies et communication entre services

05 Organisation, répartition des tâches et planning

06 Risques, perspectives

07 Conclusion

01 Présentation du projet et problématique

Problématique

Comment aider les personnes en quête de repos ou d’une expérience unique à trouver un lieu calme proche de la nature, tout en donnant de la visibilité aux maisons d’hôtes isolées ?

01 Présentation du projet et problématique

Présentation du projet

StayEasy est une plateforme web qui regroupe des maisons d’hôtes pour proposer des séjours de repos et de bien-être. Les clients choisissent un lieu calme, réservent, paient en ligne et peuvent rejoindre une maison partagée.

01 Présentation du projet et problématique

Palette des couleurs

Logo

STAYEASY

HÉBERGEMENT & COLOCATION

Palette des couleurs

Violet foncé

Confiance et sérieux : un service fiable à qui l'on confie son logement.

Violet orchidée

Bienveillance et convivialité : l'accueil et le partage de la colocation.

Turquoise

Sérénité et simplicité

02 Acteurs et fonctionnalités principales

01

Voyageur/Etudiant

Trouver une chambre, réserver et suivre son séjour.

02

Propriétaire

Gérer les chambres,les hebergements les réservations et les annonces.

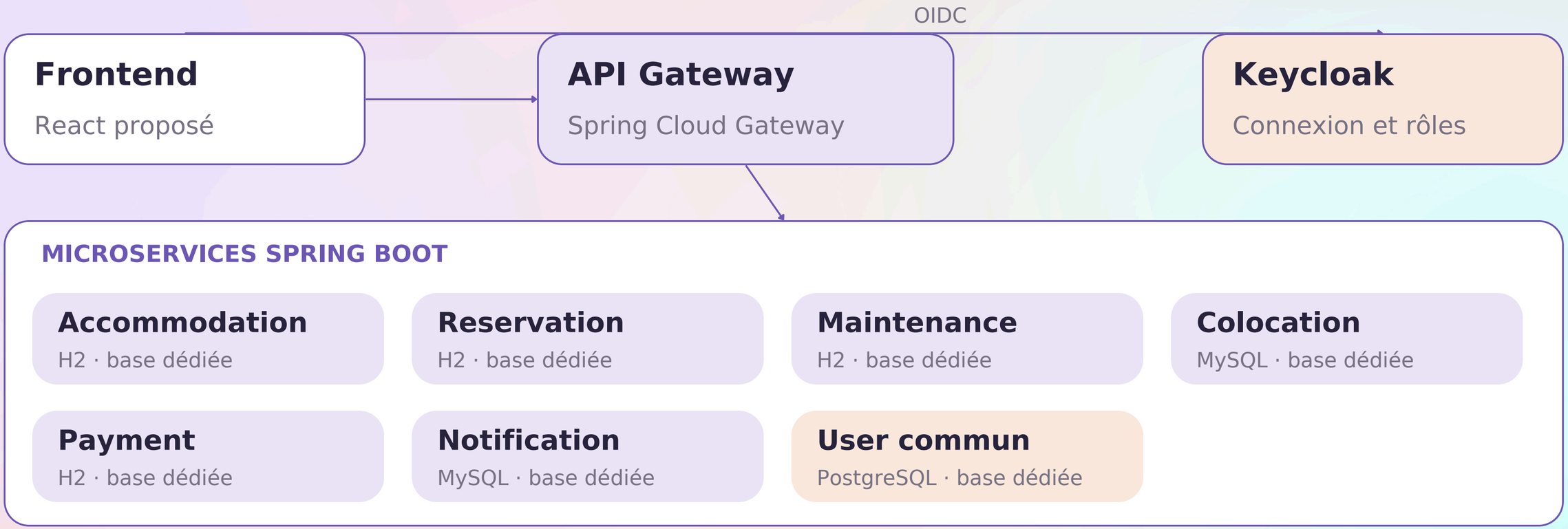
03

Équipe technique

Suivre les interventions et l’état opérationnel des chambres.

03 Architecture et microservices

1– Architecture globale



Eureka : découverte **RabbitMQ : événements** **Docker Compose**

03 Architecture et microservices

2- Digramme de classe

2– Microservices

Les responsabilités des services

Service	Responsabilité	Base / port
Hébergements	Hébergements et chambres	H2 / 8081
Réservations	Réservations et calendrier	H2 / 8082
Maintenance	Interventions techniques	H2 / 8083
Colocation	Annonces et candidatures	MySQL / 8084
Paiement	Paiements simulés	H2 / 8085
Notification	Messages et historique	MySQL / 8086
User (commun)	Profils métier	PostgreSQL / 8087

CONTRATS API REST

Méthode	Route	Description	Codes
POST	/api/users	Créer un utilisateur	201, 400, 409
GET	/api/users/{id}	Détails d'un utilisateur	200, 404
GET	/api/users/keycloak/{keycloakId}	Retrouver par keycloakId	200, 404
PUT	/api/users/{id}	Modifier un utilisateur	200, 400, 404
GET	/api/users/{id}/profil	Consulter le profil	200, 404
PUT	/api/users/{id}/profil-voyageur	Màj profil voyageur	200, 400, 404
PUT	/api/users/{id}/profil-proprietaire	Màj profil propriétaire	200, 400, 404
PUT	/api/users/{id}/statut	Activer / suspendre	200, 400, 404

POST /api/users

```
{
  "keycloakId": "a1b2-...",
  "nom": "Ben Ali",
  "prenom": "Karim",
  "email": "karim@mail.com",
  "role": "VOYAGEUR_ETUDIANT"
}
```

201 Créé

```
{
  "id": 1,
  "role": "VOYAGEUR_ETUDIANT",
  "statut": "ACTIF"
}
```

MICROSERVICE
Accomodation

Maisons d'hôtes & chambres

Catalogue d'hébergements

Créez et gérez les maisons d'hôtes et leurs informations.

Inventaire des chambres

Ajoutez les chambres avec capacité et prix par nuit.

Statut des chambres

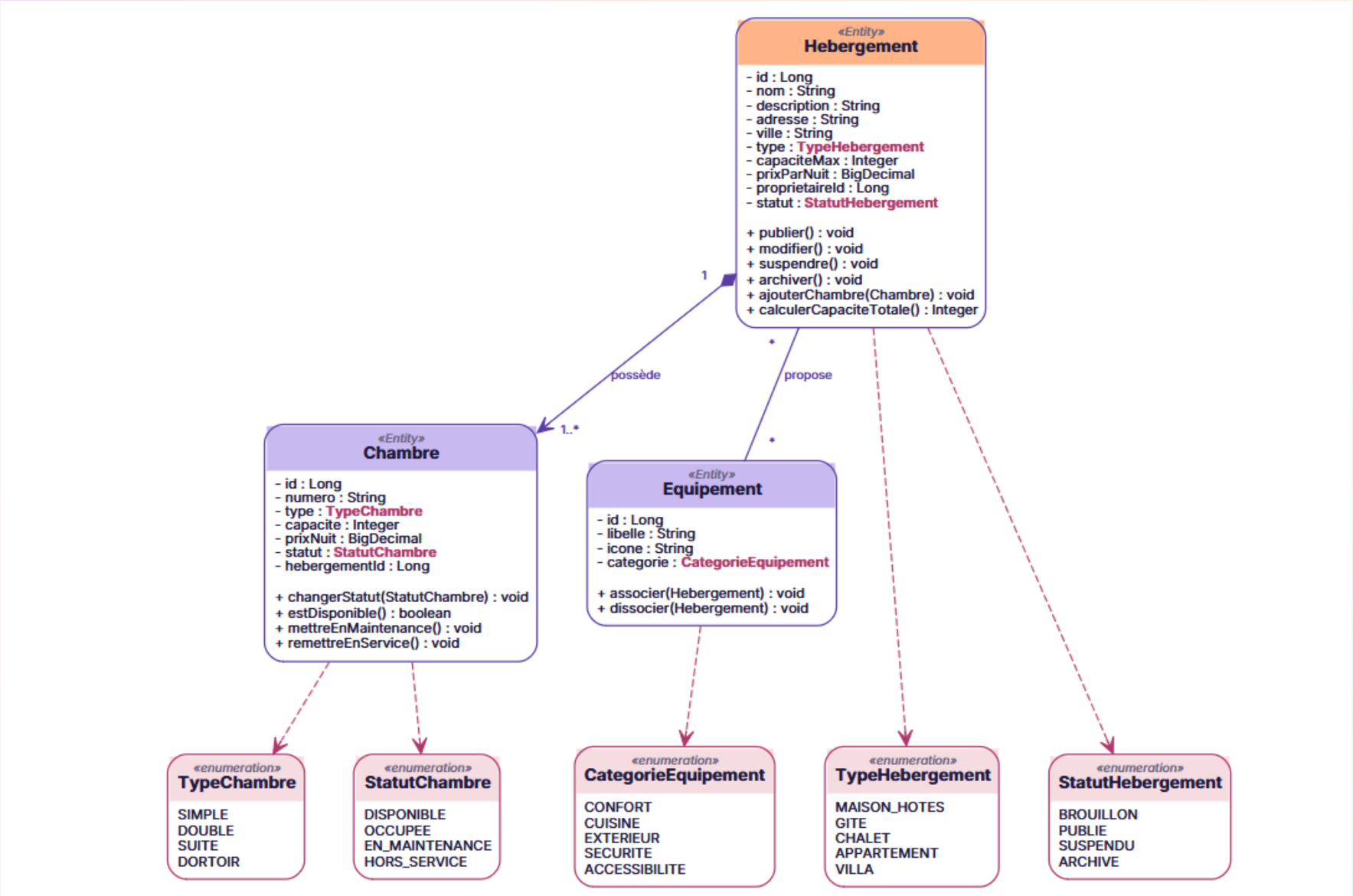
Disponible, en maintenance, hors service ou partagée.

Recherche par critères

Filtrez les hébergements par ville et par type.

DIAGRAMME DE CLASSES

Accommodation



Communication inter-microservices

↔ Synchrone : REST / OpenFeign. Le service Hébergement n'appelle aucun autre service : ce sont les autres qui l'interrogent.

MS2 · Réservation
Lit le prix, la capacité et le statut.

MS3 · Maintenance
Bloque la chambre, puis la libère.

MS4 · Colocation
Vérifie la chambre et la passe en « partagée ».

REST / OpenFeign

MS1

Accommodation

Panne signalée → **Chambre en maintenance** → **Réparation terminée** → **Chambre disponible**

CONTRATS API REST

Méthode	Route	Description	Codes
POST	/api/accommodations	Créer un hébergement	201, 400
GET	/api/accommodations?city=&type=	Lister et filtrer	200
GET	/api/accommodations/{id}	Détails d'un hébergement	200, 404
PUT	/api/accommodations/{id}	Modifier un hébergement	200, 400, 404
DELETE	/api/accommodations/{id}	Supprimer un hébergement	204, 404, 409
POST	/api/accommodations/{id}/rooms	Ajouter une chambre	201, 400, 404
GET	/api/accommodations/{id}/rooms	Lister les chambres	200, 404
GET	/api/rooms/{id}	Détails d'une chambre	200, 404
PUT	/api/rooms/{id}/status	Changer le statut	200, 400, 404

POST /api/accommodations

```
{
  "name": "Les Oliviers",
  "city": "Tabarka",
  "type": "MAISON_HOTES",
  "ownerName": "Karim"
}
```

201 Créé

```
{
  "id": 1,
  "name": "Les Oliviers",
  "status": "ACTIVE"
}
```

STAYEASY · ARCHITECTURE DES SERVICES

MICROSERVICE RESERVATION

Des séjours réservés en toute simplicité

Au cœur de StayEasy, le microservice Reservation accompagne chaque réservation, de sa création à sa confirmation, sa modification ou son annulation.

03 Architecture et microservices

Microservice Reservation

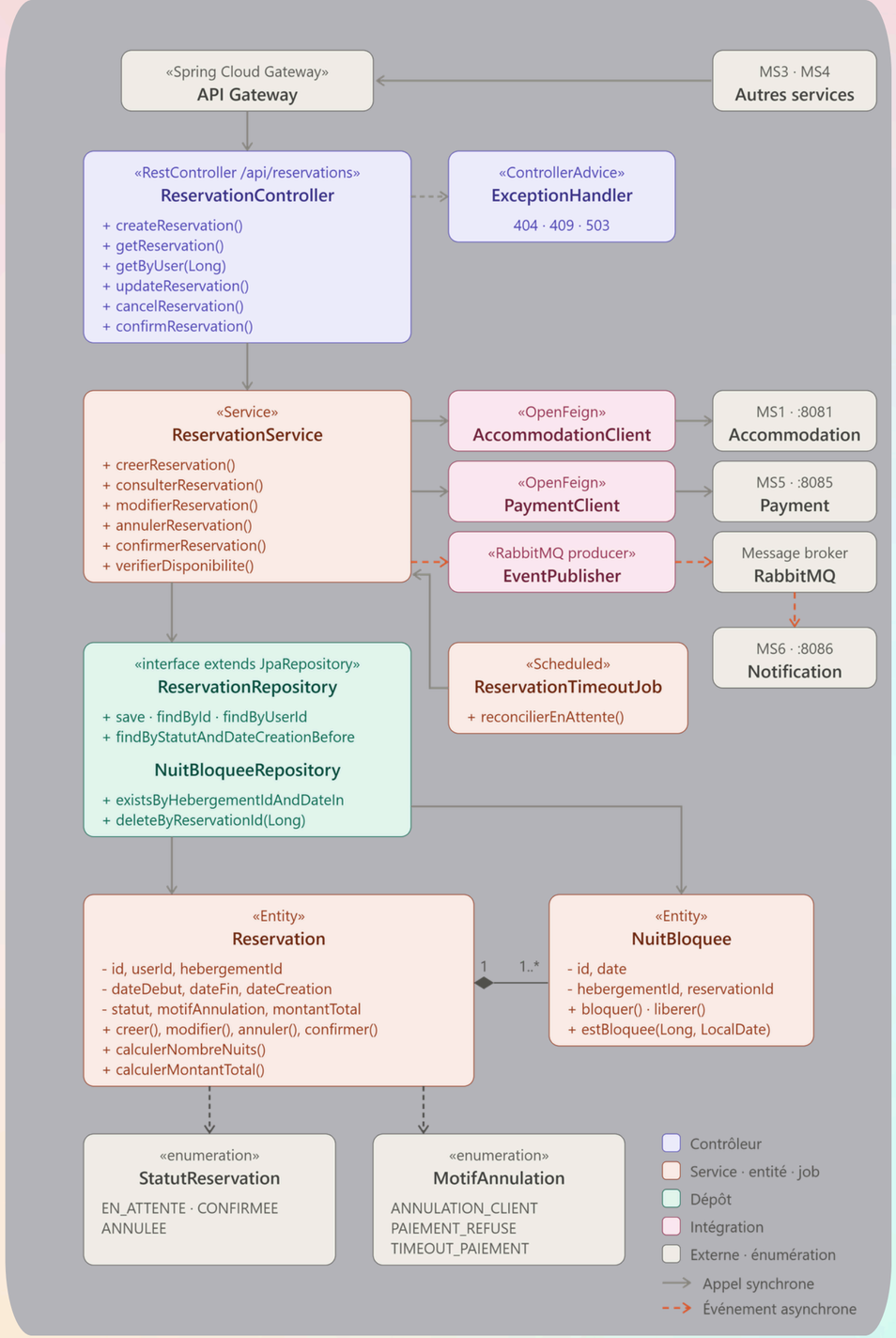
Le microservice Reservation gère le cycle de vie des réservations sur StayEasy.

1. Rôle du microservice

RÔLE	PORT DE SERVICE	BASE DE DONNÉES
MS2	8082	H2

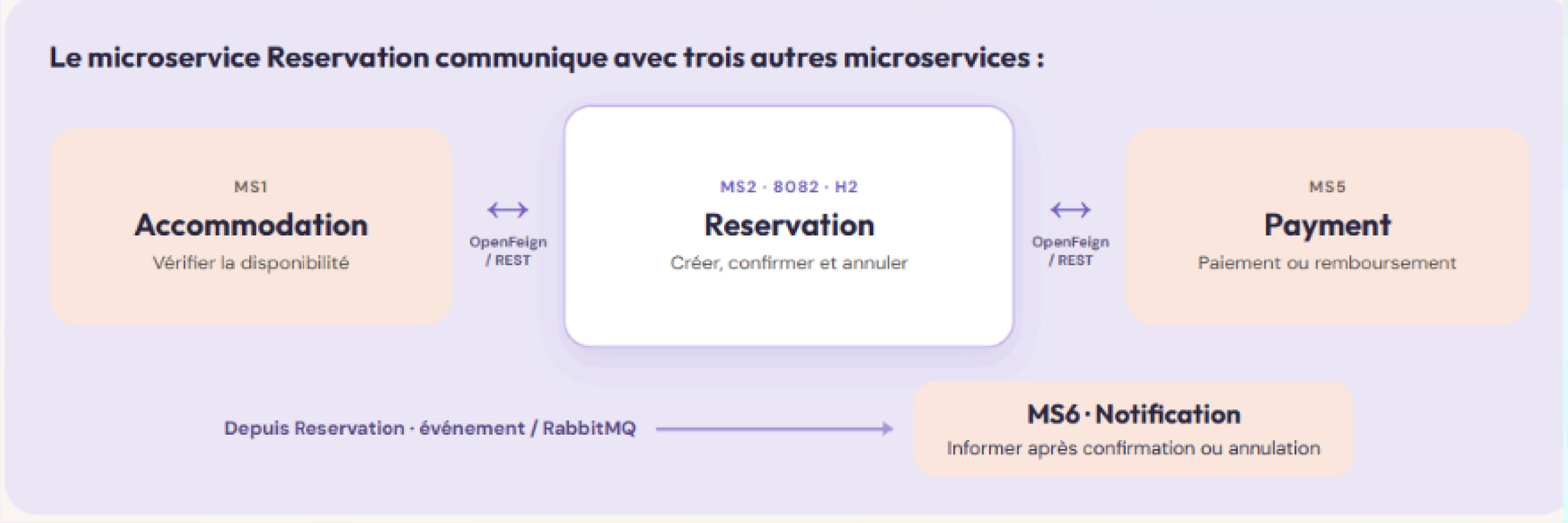
2. Diagramme de classes proposé

DIAGRAMME DE CLASSE – RESERVATION



Microservice Reservation

3-Interactions avec les autres microservices



Synchrone

OpenFeign / REST

Reservation échange directement avec **Accommodation** pour vérifier la disponibilité et avec **Payment** pour le paiement ou le remboursement.

Asynchrone

RabbitMQ

Reservation publie un événement après une confirmation ou une annulation. **Notification** le reçoit pour envoyer le message correspondant.

En bref : MS2 orchestre le parcours de réservation sur le port **8082** et conserve ses données dans **H2**.

Microservice Reservation

Le microservice Reservation gère le cycle de vie des réservations sur StayEasy.

4. Contrats d'API

Méthode	Endpoint	Fonction
POST	**/api/reservations**	**Créer une réservation**
GET	**/api/reservations/{id}**	**Consulter une réservation**
GET	**/api/reservations/user/{userId}**	**Consulter les réservations d'un utilisateur**
PUT	**/api/reservations/{id}**	**Modifier une réservation**
PATCH	**/api/reservations/{id}/confirm**	**pour la confirmer**
PATCH	**/api/reservations/{id}/cancel**	**Annuler une réservation**

Principaux codes de réponse :

CODE	SIGNIFICATION
200	Requête réussie
201	Réservation créée
400	Données invalides
404	Réservation ou ressource introuvable
409	Chambre indisponible ou conflit
500	Erreur interne du serveur
503	Dépendance indisponible

Microservice Reservation

Le microservice Reservation gère le cycle de vie des réservations sur StayEasy.

4. Contrats d'API

Événements possibles :

Événement	Déclenchement
reservation.created	Création d'une réservation
reservation.confirmed	Confirmation après paiement
reservation.cancelled	Annulation d'une réservation

MICROSERVICE COLOCATION

Hébergements partagés & expériences

Créer des séjours partagés, rapprocher les participants compatibles et organiser des activités ensemble.

GESTION DE GROUPE

Créez ou rejoignez un séjour partagé et gérez facilement les participants et leurs demandes.

MATCHING INTELLIGENT

Analysez les préférences et recommandez les groupes les plus compatibles grâce à un score

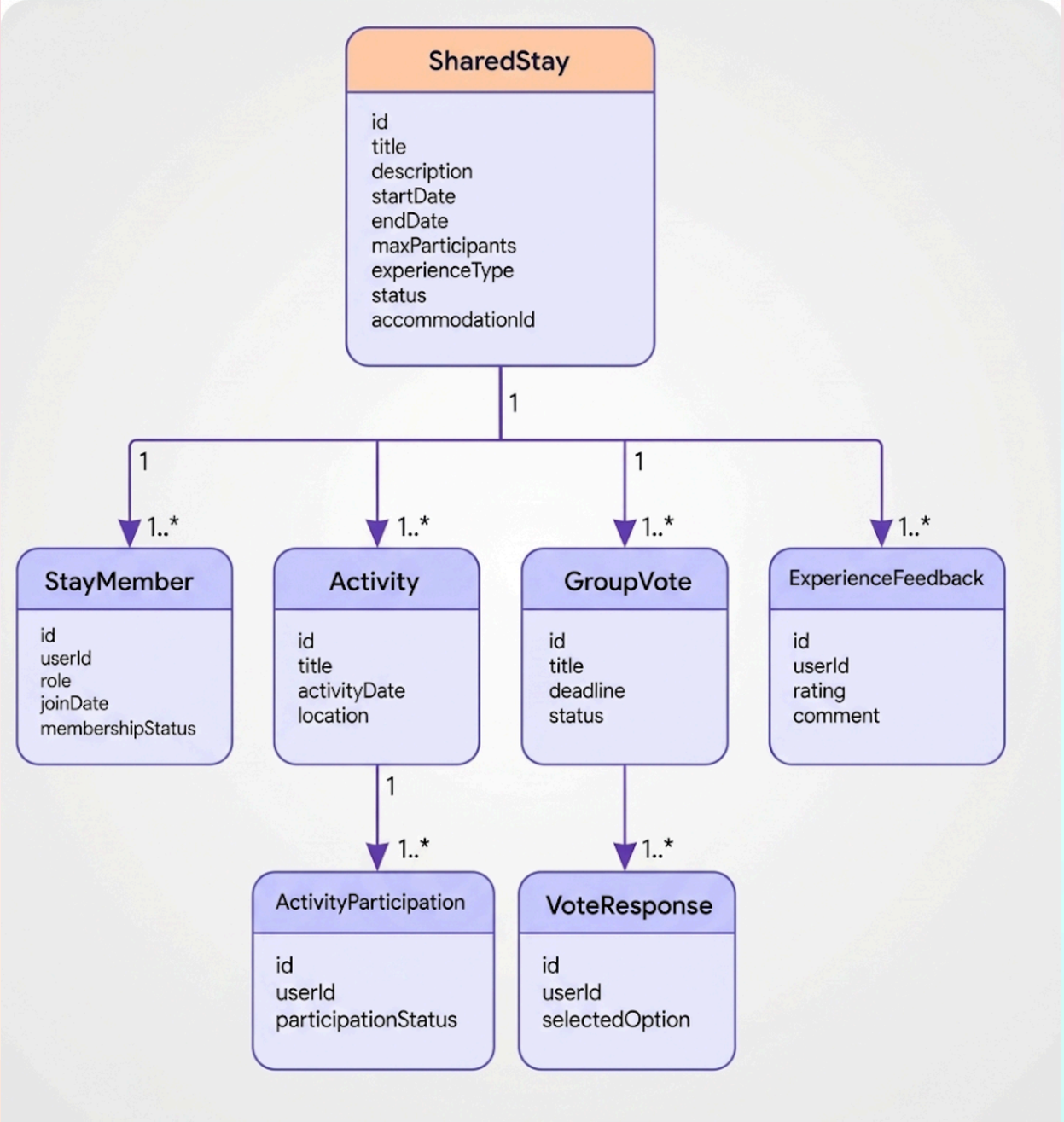
PLANIFICATION D'ACTIVITÉS

Créez des activités de groupe, organisez le calendrier et gérez les inscriptions.

COMMUNAUTÉ & INTERACTION

Échangez, votez ensemble et partagez vos retours sur l'expérience vécue.

DIAGRAMME DE CLASSE – COLOCATION

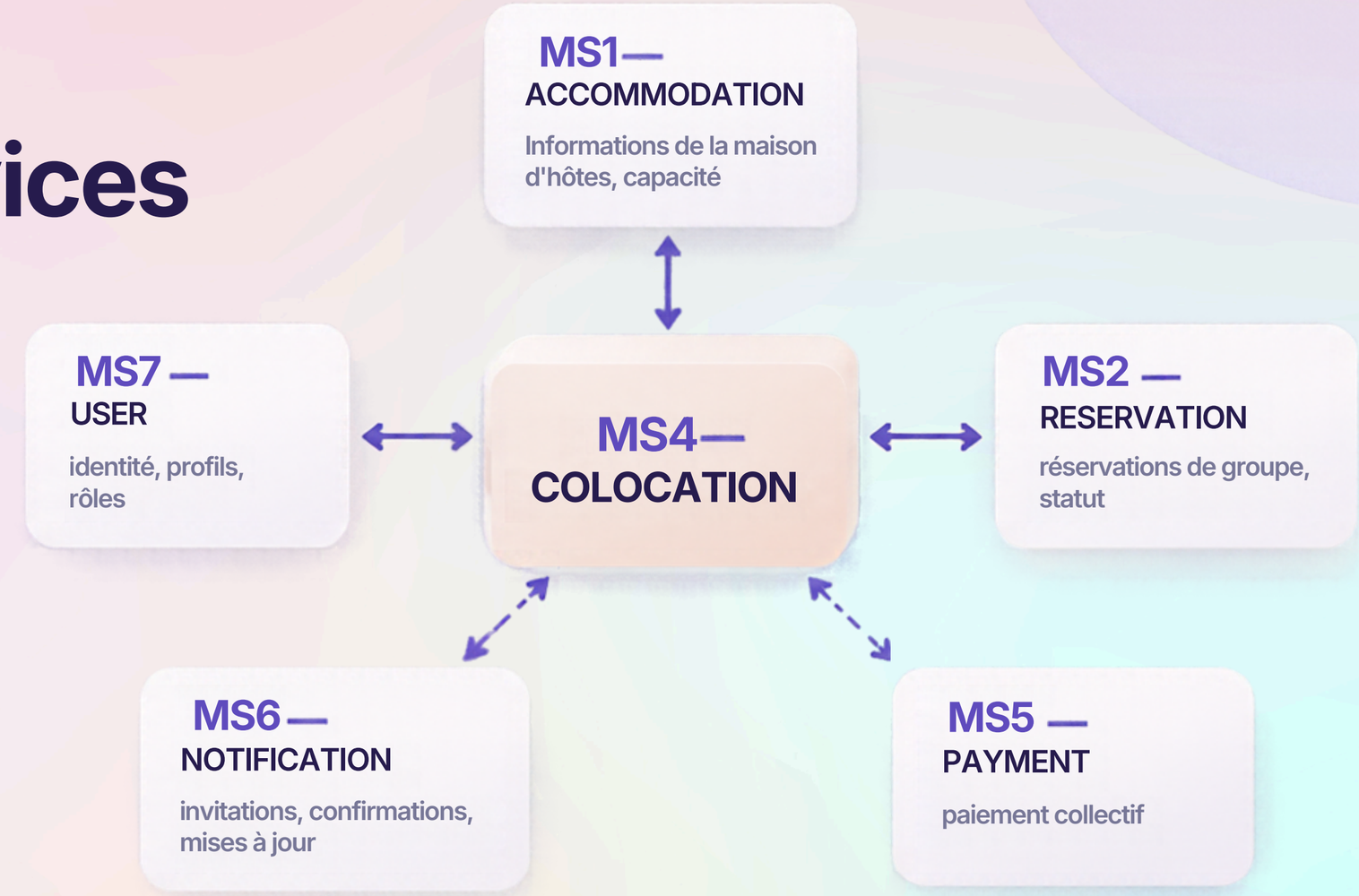


Communication Inter - Microservices

Synchrone :
REST API / HTTP

Asynchrone :
Events / Message Broker

Échange de données :
JSON



Créer un séjour partagé → Vérifier l'hébergement → Vérifier les participants → Coordonner la réservation → Traiter le paiement → Envoyer les notifications

CONTRATS API REST

Méthode HTTP	Endpoint	Description
POST	/api/v1/shared-stays	Créer un séjour partagé.
GET	/api/v1/shared-stays	Lister les séjours partagés.
GET	/api/v1/shared-stays/id}	Récupérer les détails d'un séjour partagé.
POST	/api/v1/shared-stays/id/join	Demander à rejoindre un groupe.
GET	/api/v1/shared-stays/id/members	Récupérer les participants.
POST	/api/v1/matching/calculate	Calculer la compatibilité.
POST	/api/v1/activities	Créer une activité de groupe.
POST	/api/v1/votes	Créer un vote de groupe.
POST	/api/v1/feedback	Soutmettre un retour d'expérience.

Exemple d'API – Créer un séjour partagé

POST /api/v1/shared-stays

```
{
    "title": "Nature Escape",
    "startDate": "2026-12-15",
    "endDate": "2026-12-20",
    "maxParticipants": 5,
    "experienceType": "NATURE",
    "accommodationId": 12

}
```

201 CREATED

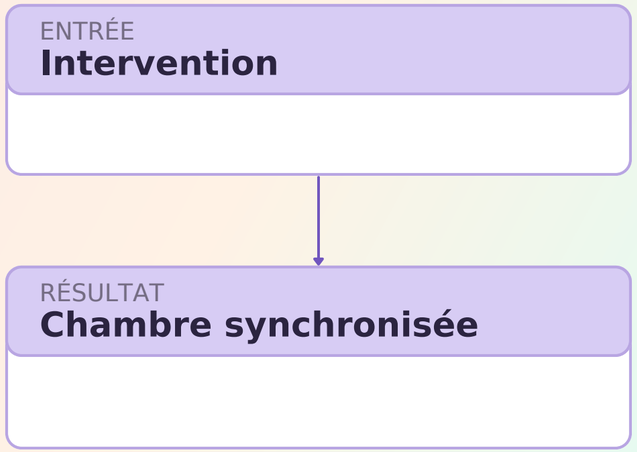
```
{
    "id": 101,
    "title": "Nature Escape",
    "currentParticipants": 1,
    "status": "OPEN"

}
```

MICROSERVICE

MAINTENANCE

Gérer les interventions techniques
et synchroniser l’état des chambres.



MS3 · Port 8083 · H2

Microservice Maintenance

Rôle du microservice

Gérer les incidents techniques et suivre leur résolution.

01 · Déclarer

Créer une intervention pour
une chambre et définir sa priorité.

02 · Traiter

Attribuer un technicien et suivre l’avancement de l’intervention.

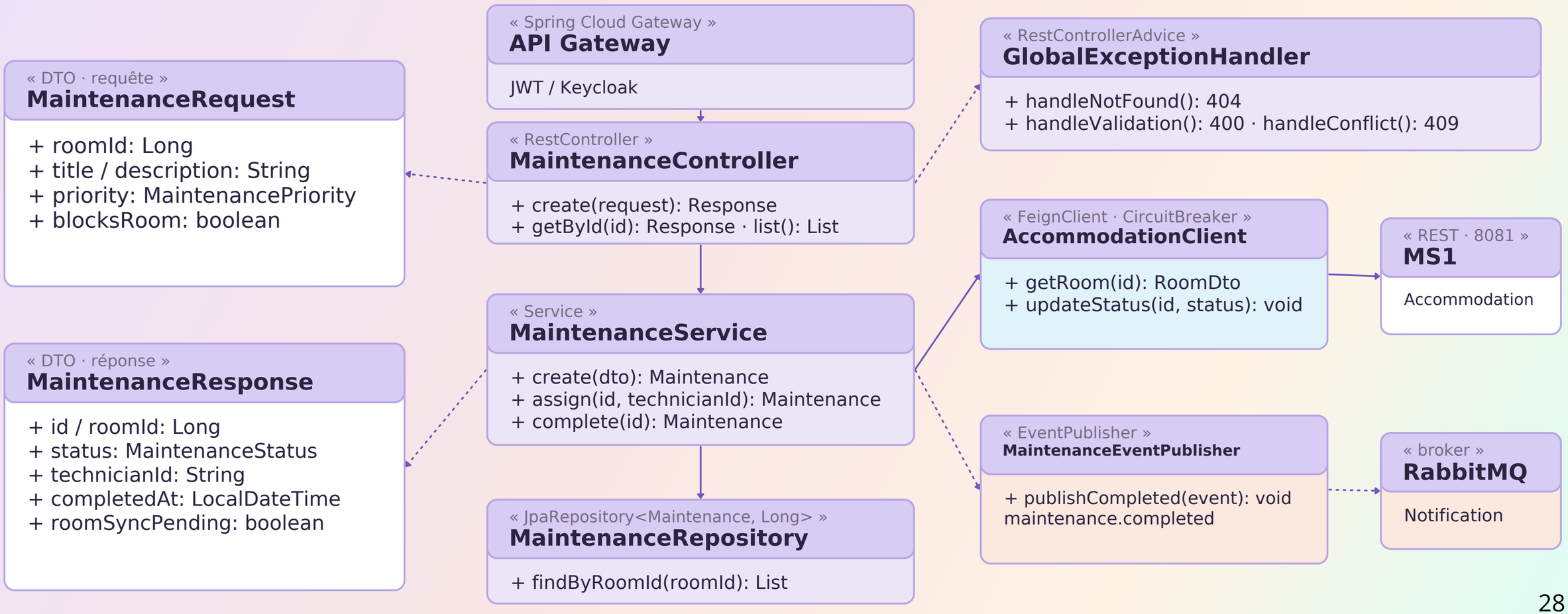
03 · Informer

Synchroniser l’état technique
avec Accommodation et publier la clôture.

SERVICE	PORT	BASE DÉDIÉE
MS3	**8083**	**H2**

Microservice Maintenance

Diagramme de classes · structure applicative



Microservice Maintenance

Diagramme de classes proposé : données métier

« Entity · base H2 »
Maintenance

id : Long · roomId : Long
title : String · description : String
priority : MaintenancePriority
status : MaintenanceStatus
technicianId : String
reportedAt / completedAt : LocalDateTime
blocksRoom : boolean
roomSyncPending : boolean

« Enum »
MaintenanceStatus

OPEN · IN_PROGRESS
COMPLETED · CANCELLED

« Enum »
MaintenancePriority

LOW · MEDIUM · HIGH

roomId référence une chambre gérée par Accommodation.

Microservice Maintenance

Interactions avec les autres microservices

Conception : aucun accès direct aux bases des autres services.

Microservice Maintenance

Contrats d’API proposés

Méthode	Endpoint	Fonction
POST	/api/maintenances	Déclarer une intervention
GET	/api/maintenances	Lister et filtrer
GET	/api/maintenances/{id}	Consulter le détail
PATCH	/api/maintenances/{id}/assignment	Attribuer un technicien
PATCH	/api/maintenances/{id}/status	Changer le statut

Filtres : roomId, status, priority

Réponses : 200 / 201 succès · 400 invalide · 404 absent · 409 conflit

Microservice Maintenance

Événements métier et notifications

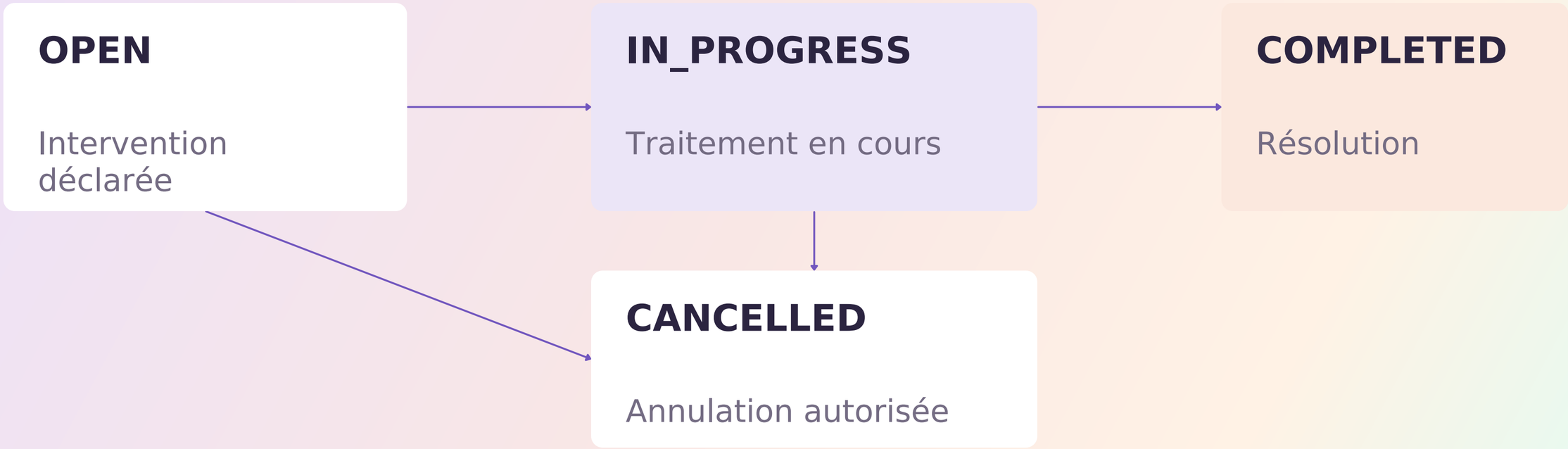
Événement	Déclenchement	Périmètre
maintenance.completed	Clôture réussie de l’intervention	Prévu
maintenance.created	Déclaration d’un problème	Optionnel
maintenance.assigned	Attribution d’un technicien	Optionnel

Contenu minimal du message

eventId, type, occurredAt, maintenanceId, roomId et destinataire autorisé

Microservice Maintenance

Cycle de vie et synchronisation de la chambre

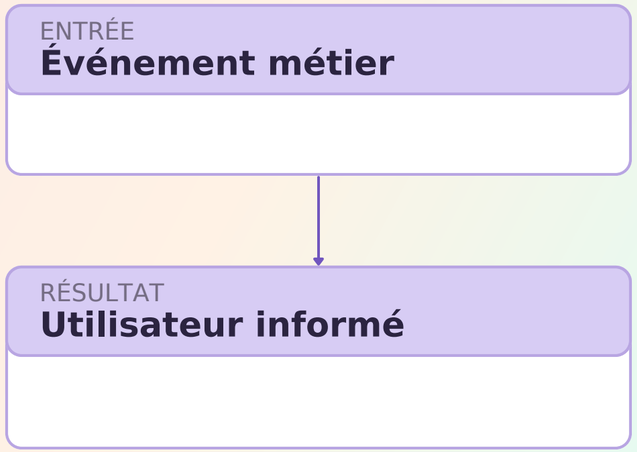


Accommodation indisponible après enregistrement : synchronisation en attente.

MICROSERVICE

NOTIFICATION

Recevoir les événements, préparer les messages et conserver l’historique des notifications.



MS6 · Port 8086 · MySQL

Microservice Notification

Rôle du microservice

Informer les utilisateurs sans bloquer les opérations métier.

01 · Recevoir

Consommer les événements de réservation, paiement et maintenance.

02 · Préparer

Identifier le destinataire et construire le message selon le type d’événement.

03 · Conserver

Enregistrer la notification, son résultat d’envoi et sa date de lecture.

SERVICE
MS6

PORT
8086

BASE DÉDIÉE
MySQL

Microservice Notification

Diagramme de classes · structure applicative

« DTO · réponse »
NotificationResponse

+ id: Long · type: String
+ title / message: String
+ status: NotificationStatus
+ createdAt / readAt: LocalDateTime

« RestController »
NotificationController

+ list(): List<NotificationResponse>
+ getById(id): NotificationResponse
+ markAsRead(id): void

« RestControllerAdvice »
GlobalExceptionHandler

+ handleNotFound(): 404
+ handleForbidden(): 403
+ handleConflict(): 409

« Service »
NotificationService

+ handleEvent(event): void
+ buildMessage(event): Notification
+ markAsRead(id): void
+ retry(id): void

« RabbitListener »
NotificationEventListener

+ onReservationEvent(event): void
+ onPaymentEvent(event): void
+ onMaintenanceEvent(event): void

« broker »
RabbitMQ

stayeasy.events

« JpaRepository<Notification, Long> »
NotificationRepository

+ existsByDeduplicationKey(key): boolean

« interface · envoi simulable »
EmailSender

+ send(notification): SendResult
Implémentation : SimulatedEmailSender

BusinessEvent

+ eventId / type: String
+ sourceService: String
+ recipientId / recipientEmail: String
+ occurredAt: LocalDateTime
+ data: Map<String, Object>

Microservice Notification

Diagramme de classes proposé : données métier

« Entity · base MySQL »
Notification

id : Long · eventId : String
recipientId : String · sourceService : String
type : String · channel : String
title : String · message : String
status : NotificationStatus
createdAt / sentAt / readAt : LocalDateTime
retryCount : int · lastError : String

« Enum »
NotificationStatus

PENDING · SENT · FAILED

« Contrainte unique »
Clé de déduplication

eventId + recipientId + channel

Le destinataire provient du contexte métier autorisé, jamais d’un accès interbase.

Microservice Notification

Contrats d’API proposés

Méthode	Endpoint	Fonction
GET	/api/notifications	Lister ses notifications
GET	/api/notifications/{id}	Consulter un message
PATCH	/api/notifications/{id}/read	Marquer comme lu
POST	/api/notifications/{id}/retry	Relancer un envoi (admin)

Filtres proposés : type, statut d’envoi et état de lecture

200 succès · 401 non connecté · 403 interdit · 404 absent · 409 conflit de relance

38

Microservice Notification

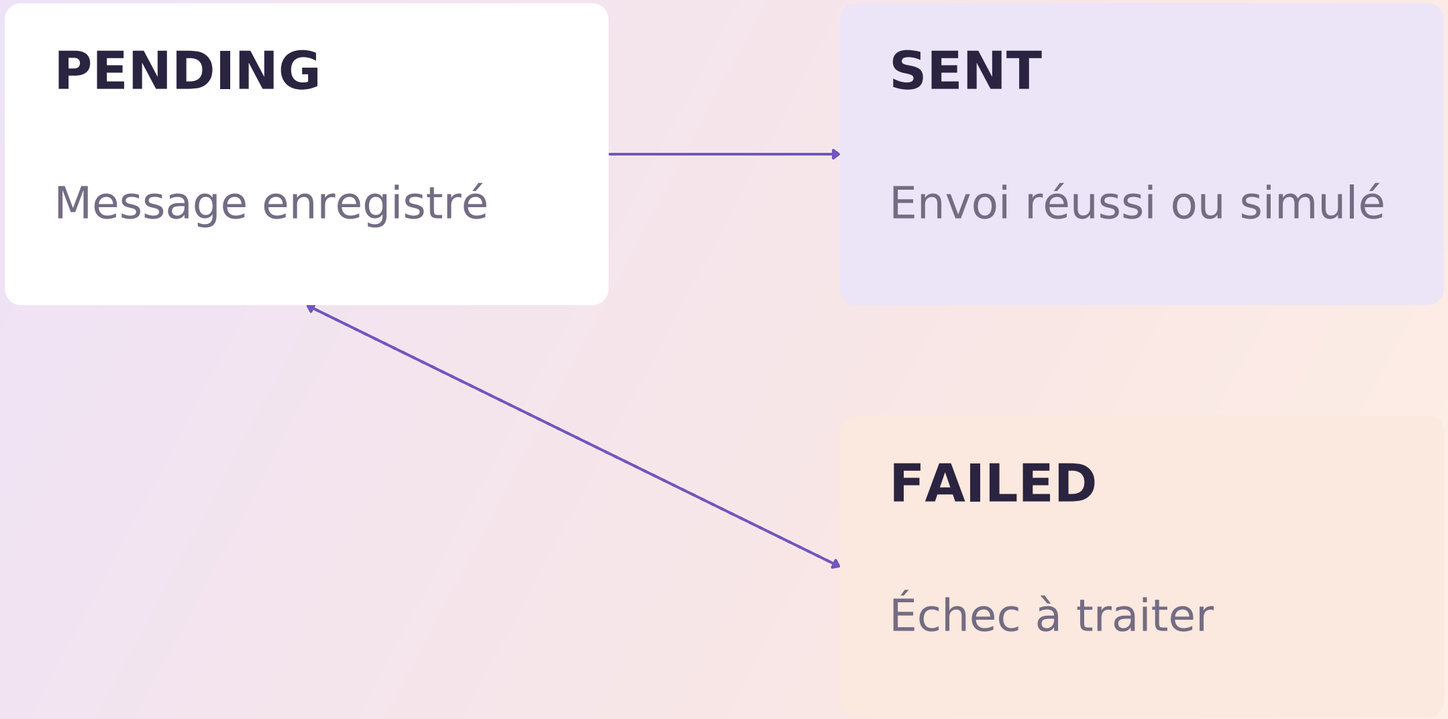
Exemples d’événements consommés

Événement reçu	Notification préparée
reservation.confirmed	Confirmation de la réservation
reservation.cancelled	Information d’annulation
payment.completed	Confirmation du paiement simulé
maintenance.completed	Information de fin d’intervention

Un événement peut produire plusieurs notifications, une par destinataire.

Microservice Notification

Traitement, reprise et prévention des doublons



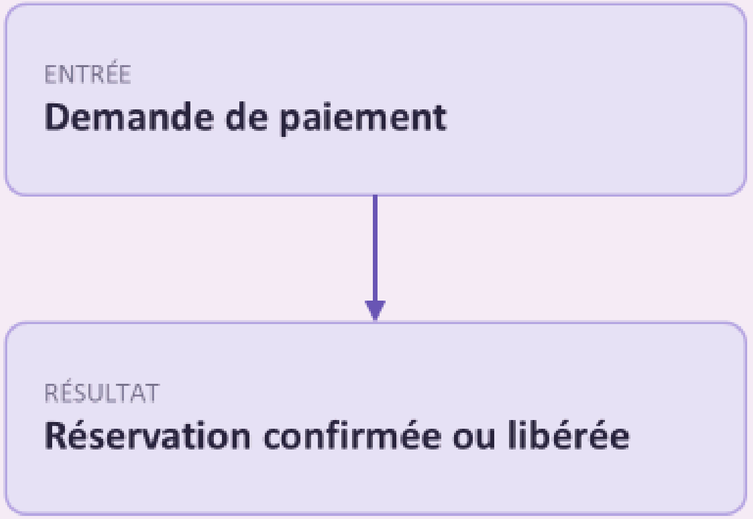
Fiabilité

Déduplication
Persistance locale
Reprises bornées
File d’erreur
Contrôle d’accès

MICROSERVICE

PAYMENT

Traiter les paiements simulés et gérer les remboursements.



MS5 · Port 8085 · H2

Microservice Payment

Rôle du microservice

Sécuriser le paiement d’une réservation et gérer son remboursement.

01 · Initier

Recevoir la demande de Reservation et vérifier le montant, la devise et la clé d’idempotence.

02 · Traiter

Simuler le paiement, enregistrer le résultat (accepté ou refusé) et gérer le remboursement.

03 · Informer

Publier payment.completed, payment.failed ou payment.refunded pour Reservation et Notification.

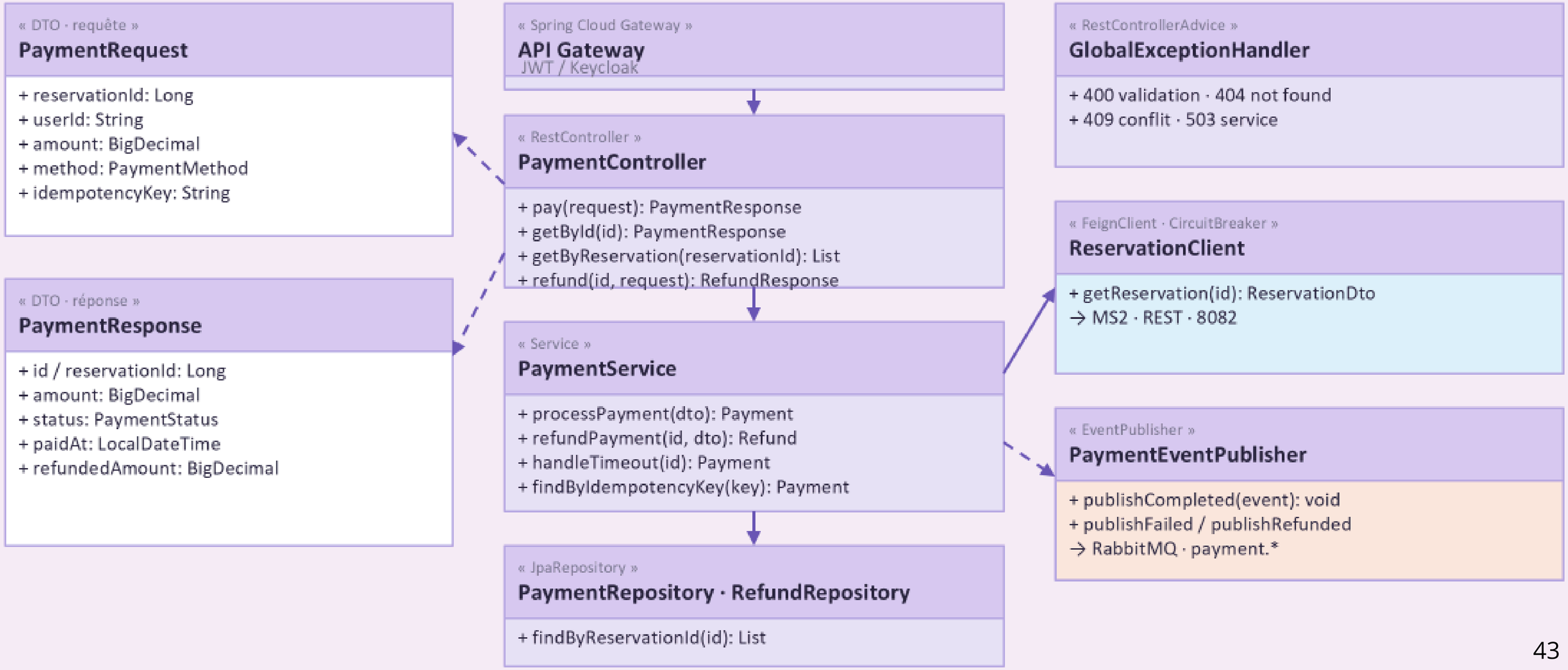
SERVICE
MS5

PORT
8085

BASE DÉDIÉE
H2

Microservice Payment

Diagramme de classes · structure applicative



Microservice Payment

Diagramme de classes proposé : données métier (2 tables)

« Entity » · table payment

Colonne	Type	Contrainte
id	BIGINT	PK, auto
reservation_id	BIGINT	NOT NULL
user_id	VARCHAR(64)	NOT NULL
amount	DECIMAL(10,2)	NOT NULL
currency	VARCHAR(3)	défaut TND
method	VARCHAR(20)	NOT NULL
status	VARCHAR(20)	NOT NULL
idempotency_key	VARCHAR(64)	UNIQUE
created_at / paid_at	TIMESTAMP	paid_at nullable

« Enum » PaymentStatus
PENDING · SUCCEEDED · FAILED · REFUNDED
« Enum » PaymentMethod : CARD_SIMULATED · CASH · TRANSFER

« Entity » · table refund

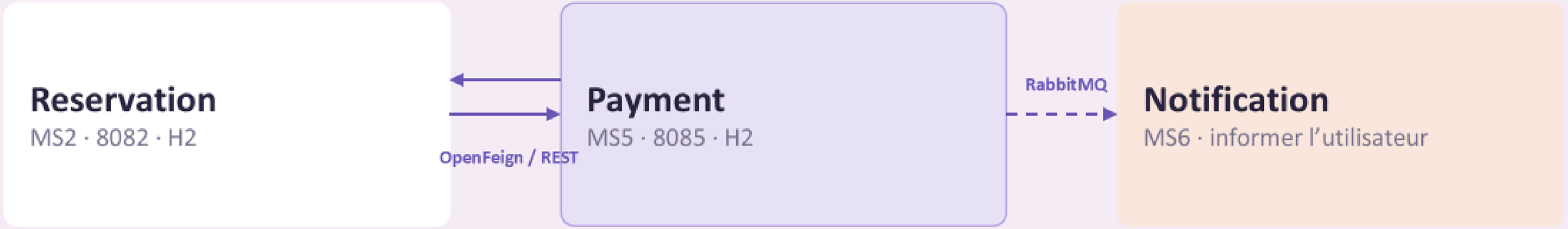
Colonne	Type	Contrainte
id	BIGINT	PK, auto
payment_id	BIGINT	FK → payment.id
amount	DECIMAL(10,2)	NOT NULL
reason	VARCHAR(255)	nullable
status	VARCHAR(20)	NOT NULL
created_at	TIMESTAMP	NOT NULL
processed_at	TIMESTAMP	nullable

1 payment — 0..* refund

reservation_id et user_id référencent d'autres services : aucune clé étrangère entre bases.

Microservice Payment

Interactions avec les autres microservices



Synchrone · OpenFeign / REST
Reservation demande le paiement ou le remboursement. Payment relit la réservation (montant, statut) via ReservationClient avec circuit breaker.

Asynchrone · RabbitMQ
Payment publie payment.completed, payment.failed et payment.refunded. Reservation confirme ou libère les dates, Notification prévient l'utilisateur.

Microservice Payment

Contrats d'API proposés

Méthode	Endpoint	Fonction
POST	/api/payments	Initier un paiement
GET	/api/payments/{id}	Consulter un paiement
GET	/api/payments/reservation/{reservationId}	Paiements d'une réservation
POST	/api/payments/{id}/refund	Rembourser un paiement
GET	/api/payments/{id}/refunds	Lister les remboursements

Principaux codes de réponse

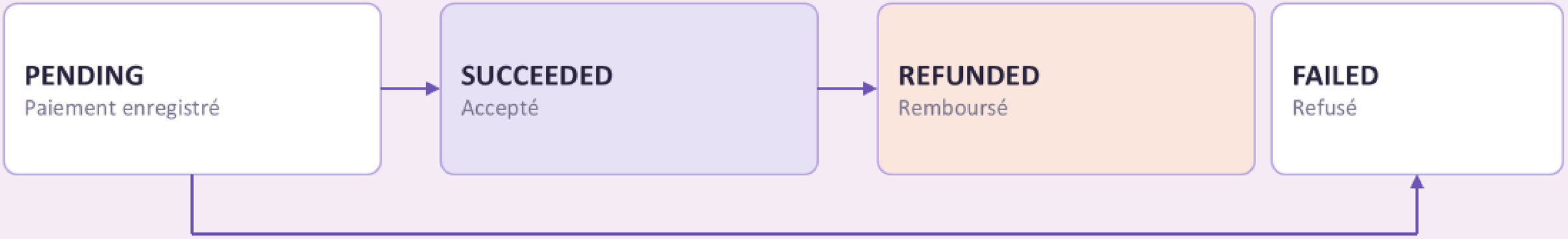
201	Paiement créé
200	Requête réussie
400	Données invalides
404	Paiement introuvable
409	Déjà payé / remboursé
503	Reservation indisponible

Filtres : reservationId, status · Idempotence : en-tête Idempotency-Key sur POST /api/payments

Microservice Payment

Événements métier et cycle de vie du paiement

Événement	Déclenchement	Consommé par
payment.completed	Paiement accepté	Reservation, Notification
payment.failed	Paiement refusé ou timeout	Reservation, Notification
payment.refunded	Remboursement effectué	Reservation, Notification



Timeout : état PENDING puis vérification du résultat avant toute annulation.

MICROSERVICE
User

Inscription et profil

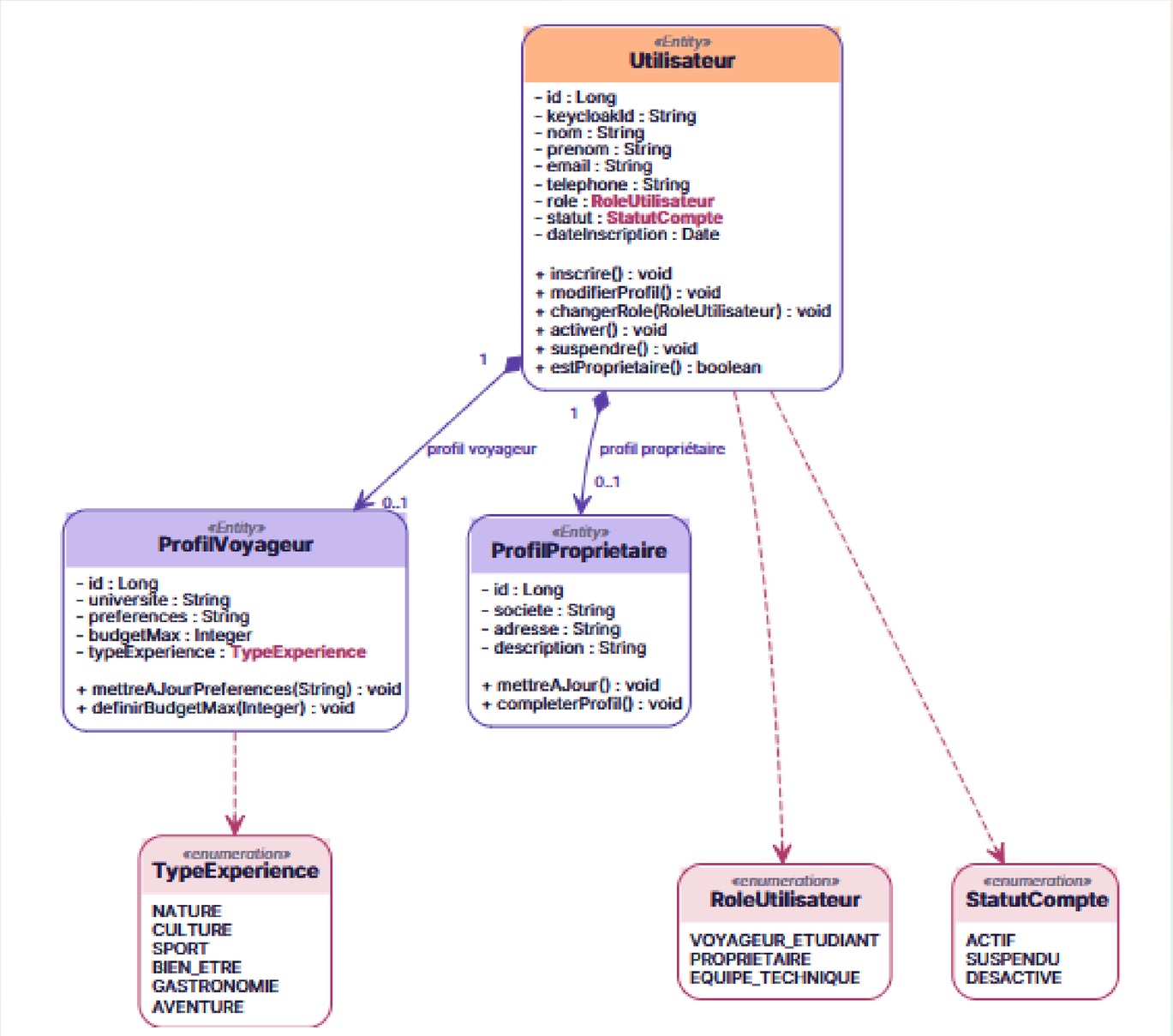
Créer le profil métier après la première connexion Keycloak.

Rôles et statut

Refléter les rôles Keycloak, activer ou suspendre un compte.

DIAGRAMME DE CLASSES

User



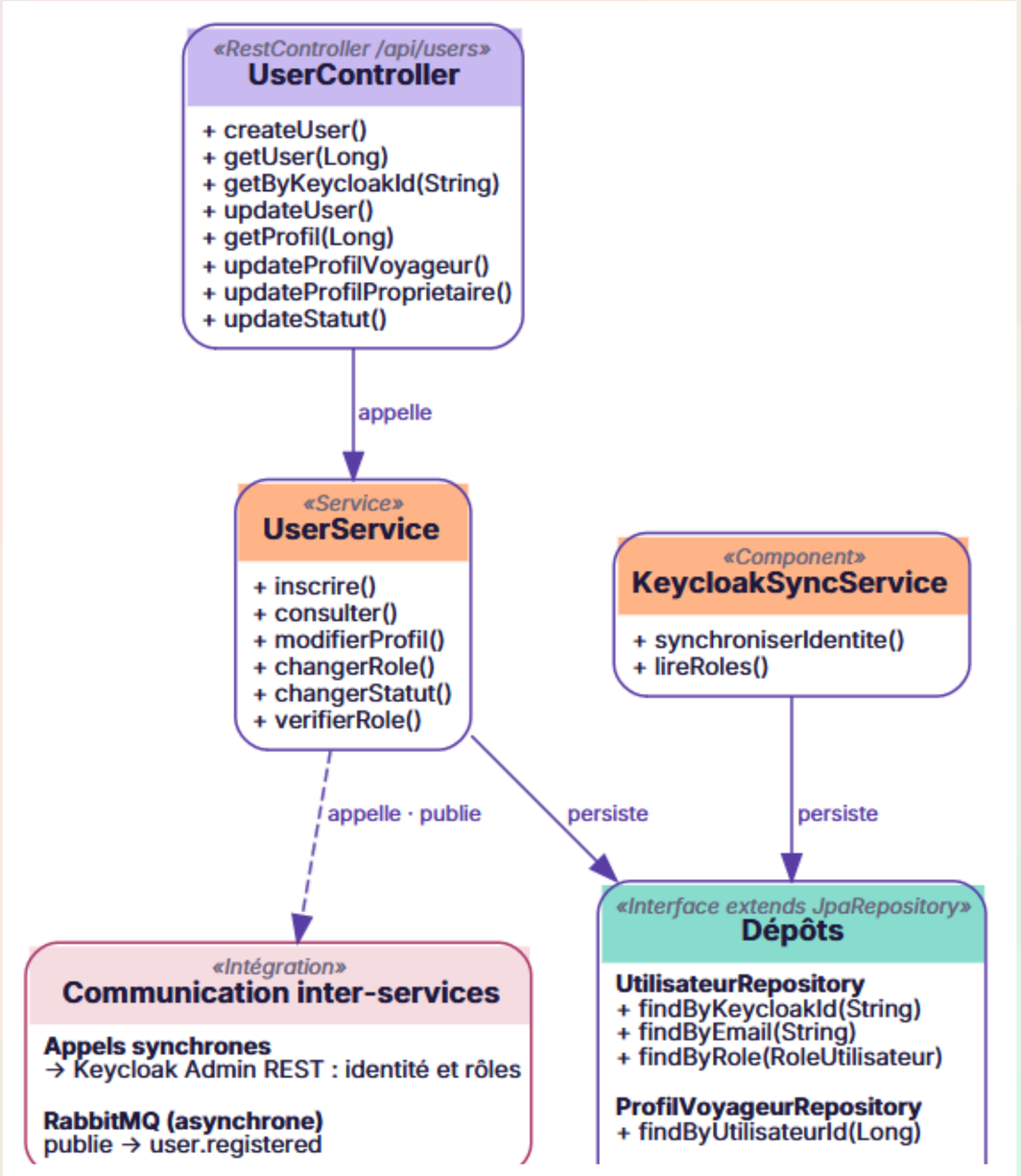
ARCHITECTURE EN COUCHES

UserController
API REST /api/users

UserService
Logique métier

KeycloakSyncService
Synchronise identité et rôles

Dépôts JPA
Utilisateur, ProfilVoyageur, ProfilProprietaire



Communication inter-microservices

Synchrone : REST / OpenFeign (Keycloak Admin REST). Asynchrone : RabbitMQ.

MS1 · Hébergement Vérifie le propriétaire.

MS2 · Réservation Vérifie le voyageur.

MS3 · Maintenance Vérifie le technicien.

MS4 · Colocation Vérifie les participants.

⟷

MS7

User

→ Keycloak Admin REST : identité et rôles
→ RabbitMQ : publie user.registered

Première connexion Keycloak → **Profil créé** → **Événement user.registered** → **Rôle vérifié par les services**

04 Technologies et communication entre services

01

Backend

Java / Spring Boot
Gateway et Eureka
Swagger / OpenAPI

02

Données et accès

H2 / MySQL / PostgreSQL
Keycloak : identité
Rôles et autorisations

03

Intégration

React proposé
RabbitMQ proposé
Docker Compose

05 Organisation, répartition des tâches et planning

La répartition du travail

Chaque étudiant

E1 : Accommodation
E2 : Reservation
E3 : Maintenance
E4 : Colocation
E5 : Payment
E6 : Notification

Tout le groupe

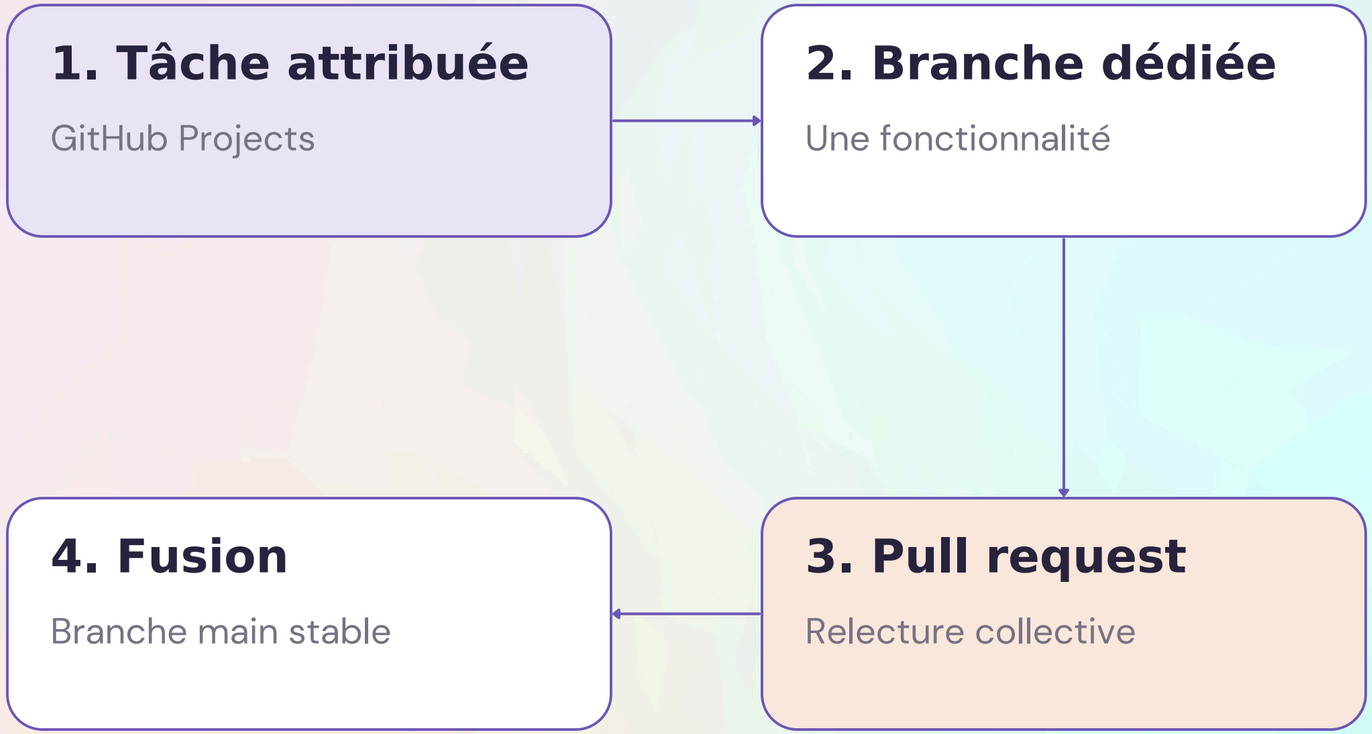
Service User
Frontend et intégration
Keycloak, Gateway et Eureka
RabbitMQ et Docker
Documentation et démonstration

05 Organisation, répartition des tâches et planning

Organisation proposée

Le dépôt StayEasy

```
Backend/
Frontend/
Documentation/
  Presentation/
  Diagrams/
README.md
```



05 Organisation, répartition des tâches et planning

De la séance 4 à la séance 10

S4

Conception

Architecture et contrats

S5/6

Développement

Services, Gateway et REST

S7/8

Intégration

Événements, sécurité et frontend

S9/10

Livraison

Docker, tests et soutenance

06

Risques, perspectives

DES MESURES PRÉVUES DÈS LA CONCEPTION

01

Double réservation

Blocage atomique des nuits.

02

Paiement incertain

Idempotence et réconciliation.

03

Message perdu

Reprise et déduplication.

04

Retard d’intégration

Contrats partagés et priorités.

06 Perspectives

APRÈS LA PREMIÈRE VERSION FONCTIONNELLE

01

Application mobile

Réserver et suivre son séjour depuis un smartphone.

02

Paiement réel sécurisé

Intégrer un prestataire de paiement et automatiser les remboursements.

03

Maintenance préventive

Planifier les contrôles des chambres pour anticiper les interventions.

04

Tableau de bord de gestion

Suivre l’occupation, les revenus et les délais de résolution.

07 Conclusion

Un séjour organisé, des responsabilités bien définies.

Le projet

Réservation et colocation

La conception

7 services et bases dédiées

La prochaine étape

Valider puis développer

MERCI DE VOTRE ATTENTION

Des questions ?

STAYEASY
HÉBERGEMENT & COLOCATION

MICROSERVICE USER

Gestion centralisée des profils et des utilisateurs.

Le microservice User stocke et gère les profils métier des utilisateurs de StayEasy. Il est commun à toute la plateforme et travaille en complémentarité avec Keycloak, qui prend en charge l'identité et l'authentification.

INSCRIPTION ET PROFIL

Créer le profil métier après la première connexion à Keycloak.

PROFILS SPÉCIALISÉS

ProfilVoyageur : université, préférences et budget.
ProfilPropriétaire : société et adresse.

RÔLES ET STATUT

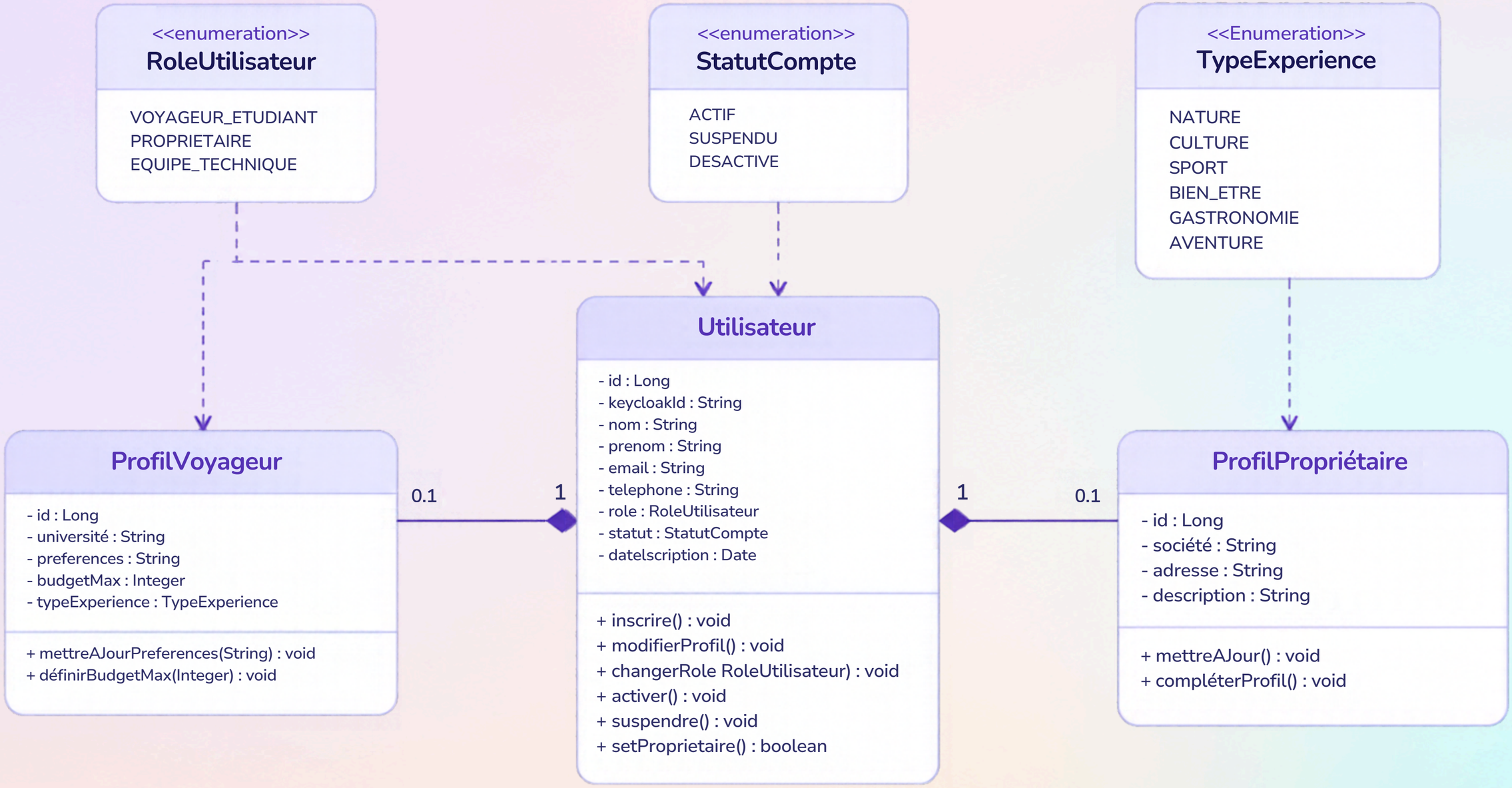
Réfléter les rôles Keycloak et gérer l'activation ou la suspension des comptes.

ANNUAIRE INTERNE

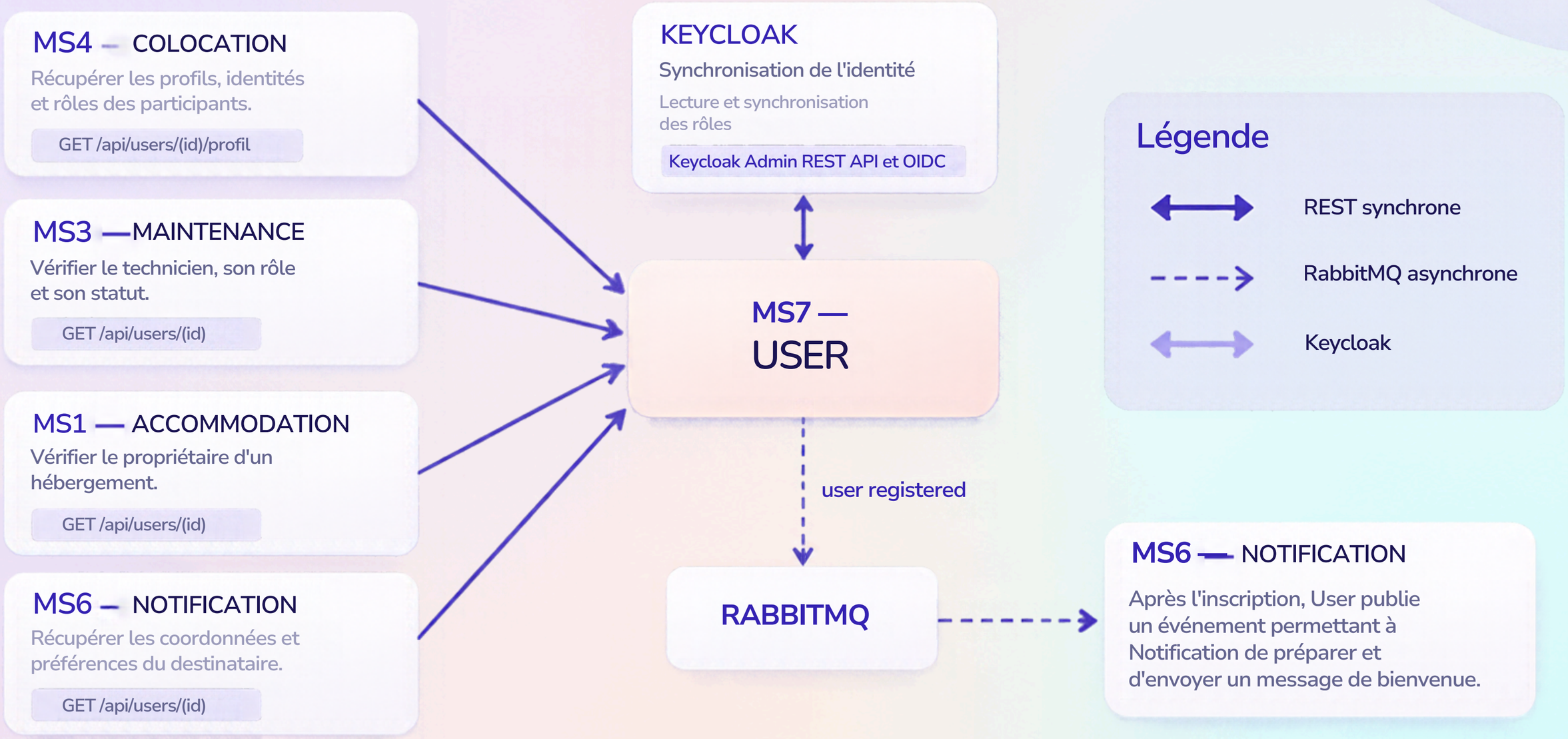
Permettre aux autres microservices de vérifier l'identité, le rôle, le statut et les coordonnées des utilisateurs.

Aucun mot de passe n'est stocké par StayEasy : l'identité reste dans Keycloak et seul le keycloakld fait le lien.

DIAGRAMME DE CLASSE – USER



Communication
Inter - Microservices



API REST – CONTRATS

Méthode HTTP	Endpoint	Description
POST	/api/users	Créer le profil après connexion.
GET	/api/users{id}	Consulter un utilisateur.
GET	/api/users/keyclock/{keyclockId}	Retrouver un utilisateur via Keyclock.
PUT	/api/users{id}	Modifier un utilisateur.
GET	/api/users{id} /profil	Consulter le profil voyageur ou propriétaire.
PUT	/api/users{id} /profil-voyageur	Mettre à jour le profil voyageur.
PUT	/api/users{id} /profil-propriétaire	Mettre à jour le profil propriétaire.
PATCH	/api/users{id}/statut	Activer ou suspendre un compte.

EXEMPLE D'API – INSCRIPTION D'UN UTILISATEUR

EXEMPLE DE REQUÊTE

POST /api/users

```
{
  "keyclockId": "a1b2-c3d4",
  "nom": "Ben Ali",
  "prenom": "Sarra",
  "email": "sarra@mail.tn",
  "role": "VOYAGEUR_ETUDIANT"
}
```

EXEMPLE DE RÉPONSE

201 CREATED

```
{
  "id": 7,
  "role": "VOYAGEUR_ETUDIANT",
  "statut": "ACTIF"
}
```

409 : e-mail ou keyclockId déjà utilisé.

403 : tentative de modification du profil d'un autre utilisateur.

RAPPEL DE SÉCURITÉ

Authentification centralisée par Keyclock.
Gestion des autorisations selon les rôles.
Aucun mot de passe enregistré dans la base PostgreSQL de User.

Un profil métier centralisé pour connecter tous les acteurs de StayEasy.